Two Revolutions: GPUs for Science, AI for Scientists

Will Handley
<wh260@cam.ac.uk>

Royal Society University Research Fellow
Institute of Astronomy, University of Cambridge
Kavli Institute for Cosmology, Cambridge
Gonville & Caius College
willhandley.co.uk/talks

20th January 2026

Two Revolutions

Distinct shifts, often conflated

Revolution 1: GPUs for Science

- GPU hardware (developed for ML) accelerates *classical* statistical methods
- Nested sampling speedups of 100–1000$\times$
- Rigorous Bayesian inference at previously impractical scales

Revolution 2: AI for Scientists

- LLMs change how we write code and manage research
- Not automating the science itself
- Reducing the technical overhead that consumes researcher time

The Argument

These are distinct. One accelerates the *calculation*; the other accelerates the *researcher*. They may be more separable than current discourse implies.

Inference Across Astrophysics

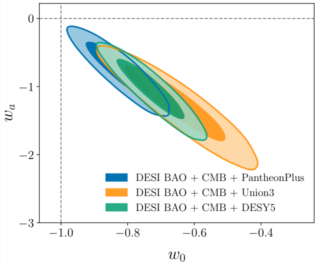
Parameter estimation and model comparison

$\mathcal{P}(\theta|D) = \frac{\mathcal{L}(D|\theta)\pi(\theta)}{\mathcal{Z}}$ **Estimate**: Compute posterior $\mathcal{P}(\theta|D)$ **Compare**: evidences $\mathcal{Z}_1/\mathcal{Z}_2$

Cosmology

Estimate: H_0, Ω_m, w

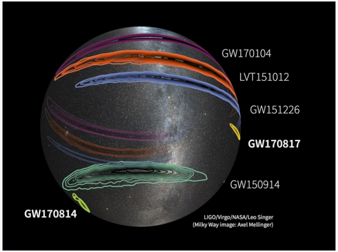
Compare: ΛCDM vs wCDM?



Gravitational Waves

Estimate: m, χ, θ, ϕ

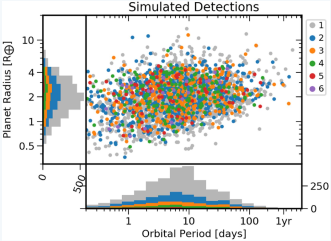
Compare: NS-NS vs BH-BH?



Exoplanets

Estimate: P, R, i

Compare: N_{planets}?



The Common Challenge: Complex Parameter Spaces

Why these problems are statistically hard

High-Dimensional

Many parameters to infer

- Cosmo: 6+ params + nuisance
- GW: 15+ parameters
- EP: $\sim$5–10/planet

Challenge: Volume grows exponentially with dimension

Degenerate

Parameters strongly correlated

- Cosmo: H_0–Ω_m banana
- GW: Mass-distance degeneracy
- EP: Eccentricity-period

Challenge: Correlated directions hard to explore

Multimodal

Multiple distinct solutions

- Cosmo: Tensions $\rightarrow$ separated modes
- GW: Bimodal inclination
- EP: Orbital period aliases

Challenge: Easy to miss plausible explanations

Key Message

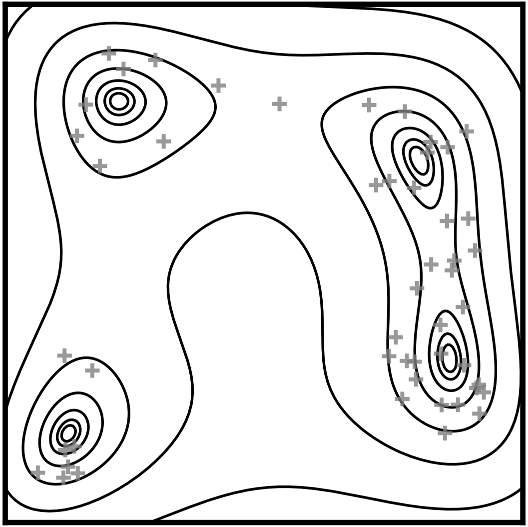
These three features make inference computationally expensive and require sophisticated statistical methods [MCMC, HMC, SMC, Nested Sampling]

Why do sampling?

- The cornerstone of numerical Bayesian inference is working with **samples**.
- Generate a set of representative parameters drawn in proportion to the posterior $\theta \sim \mathcal{P}$.
- The magic of marginalisation $\Rightarrow$ perform usual analysis on each sample in turn.
- The golden rule is **stay in samples** until the last moment before computing summary statistics/triangle plots because

$$\boxed{f(\langle X \rangle) \ne \langle f(X) \rangle}$$

- Generally need $\sim \mathcal{O}(12)$ independent samples to compute a value and error bar.



Sampling Methods: A Brief Tour

MCMC (e.g. `emcee`)

- Random walk explores posterior
- Good for simple, well-behaved posteriors
- Can get stuck in local modes
- Posterior samples only

HMC/NUTS (e.g. `stan`, `numpyro`)

- Uses gradients to guide exploration
- Much more efficient for smooth posteriors
- Requires differentiable likelihoods
- Posterior samples only

Nested Sampling (e.g. `dynesty`)

- Explores from outside-in
- Naturally handles multimodality
- Computes Bayesian Evidence
- Essential for model comparison (i.e. science)

"The evidence $\mathcal{Z}$ is often the single most important number in the problem and I think every effort should be devoted to calculating it." — MacKay (2003)

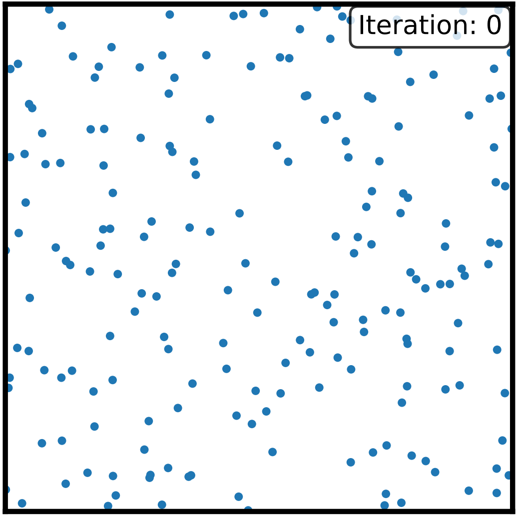
Nested Sampling

Exploring from the outside-in

A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

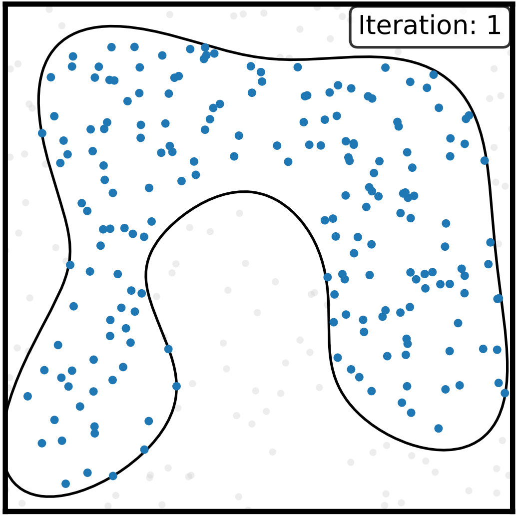
1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

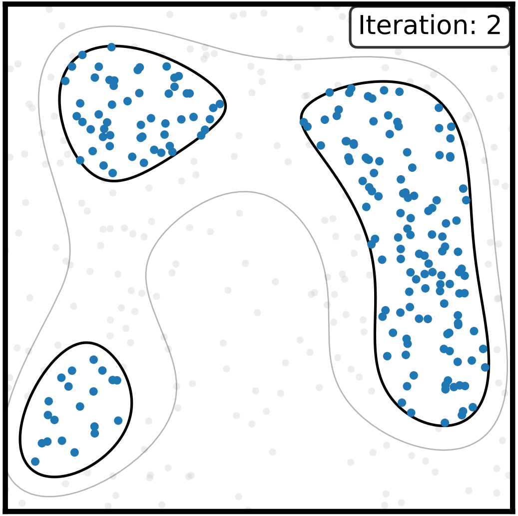
1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

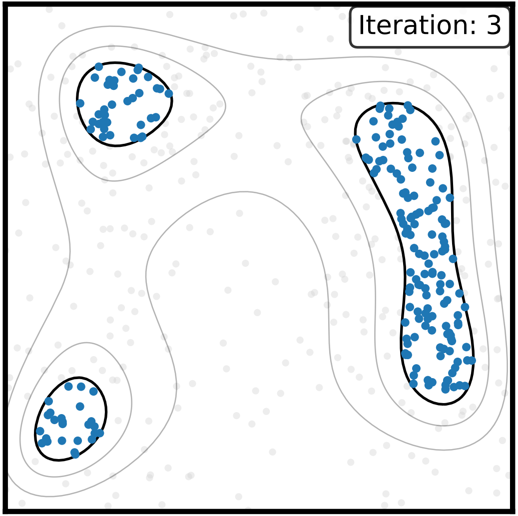
1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

1. Start with a set of “live points” scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously “shrink” into regions of higher and higher likelihood.



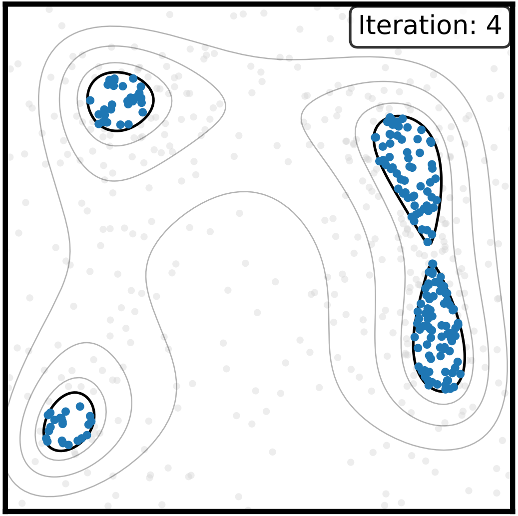
Nested Sampling

Exploring from the outside-in

A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

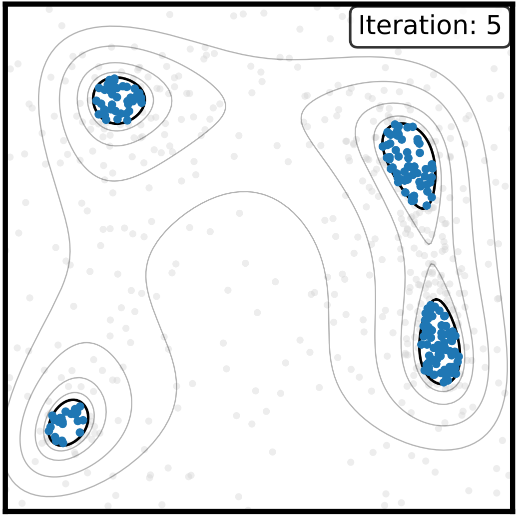
1. Start with a set of “live points” scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously “shrink” into regions of higher and higher likelihood.



A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



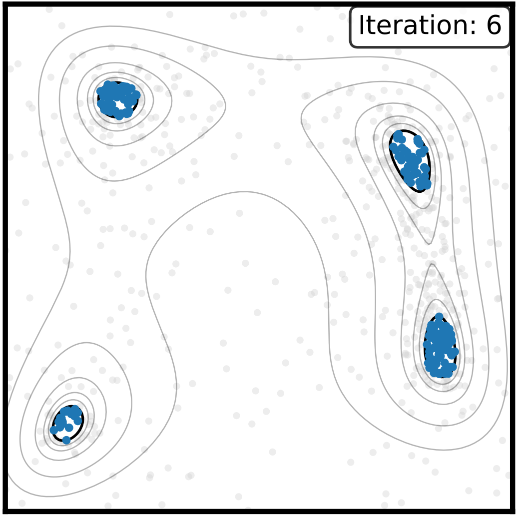
Nested Sampling

Exploring from the outside-in

A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



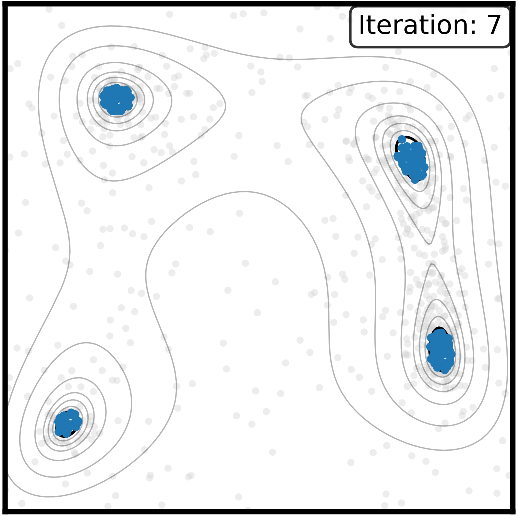
Nested Sampling

Exploring from the outside-in

A Radically Different Approach

Instead of random walking, nested sampling attacks the problem from the outside-in.

1. Start with a set of "live points" scattered across the entire **prior**.
2. At each step: find the point with the *worst* likelihood and discard it.
3. Replace it with a new point drawn from the prior, but with a likelihood *better* than the point you just discarded.
4. This forces the set of live points to continuously "shrink" into regions of higher and higher likelihood.



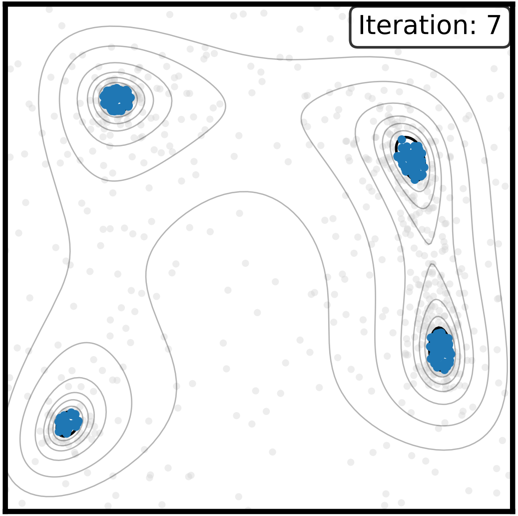
Why This Works

- Naturally handles **multimodality**: the shrinking cloud finds all modes simultaneously.
- Estimates **volume** probabilistically:

$$\frac{V_{\mathrm{after}}}{V_{\mathrm{before}}} \approx \frac{n_{\mathrm{in}}}{n_{\mathrm{out}} + n_{\mathrm{in}}}$$

- Computes evidence $\mathcal{Z}$ and **information** $\mathcal{D}_{\mathrm{KL}}$:

$$\log \mathcal{Z} = \langle \log \mathcal{L} \rangle_{\mathcal{P}} - \mathcal{D}_{\mathrm{KL}}$$

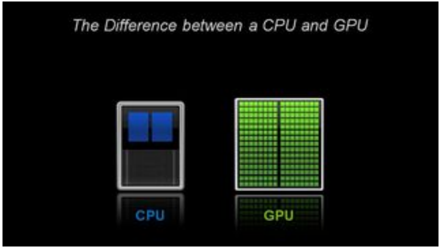


The Computational Bottleneck

Why the future of astronomical inference is parallel computing

The Problem

- Nested sampling is powerful but **slow**
- Complex parameter spaces require many likelihood evaluations
- Traditional CPU approaches struggle to scale



CPU vs GPU: Different Approaches

- **CPU**: Few powerful cores (~10s), sequential tasks
- **GPU**: Many simple cores (~1000s), parallel tasks

Which Algorithms Can Exploit This?

- Traditional MCMC/HMC: single walker → no
- `emcee`: ensemble of walkers → yes
- Nested sampling: many live points → yes

The Future is GPU

All new HPC facilities are GPU-based. We must adapt our codes.

Two Independent Capabilities

A common misconception in scientific computing

Differentiable programming languages: JAX, PyTorch, TensorFlow, Julia, Stan, ...

Capability 1: Free Gradients

- **Automatic differentiation**: $\nabla_\theta \log \mathcal{L}(\theta)$.
- Enables gradient-based MCMC (HMC, NUTS).
- Essential for modern optimization.

Traditional Physics Benefits

- **Nested sampling**: Massive parallelization.
- **21cm signals**: Vectorized across frequency/time/angle.
- **N-body sims**: GPU acceleration.

Capability 2: GPU Parallelization

- **Vectorization across ensembles**.
- Run 1000s of parallel chains/particles.
- Evaluate likelihoods simultaneously.

Key Insight: Often Confused

These are completely independent.
People mistake one for the other.
You can use gradients on CPU.
You can GPU parallelize without gradients.
They serve different purposes.

BlackJAX: GPU Native Sampling

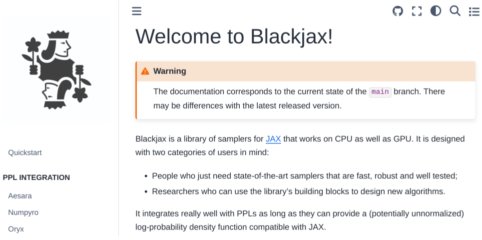
A unified framework for GPU-accelerated inference

The Challenge

- Sampling traditionally CPU-bound
- Different algorithms, same GPU challenge
- Need unified GPU-native framework

BlackJAX Solution

- Full JAX ecosystem integration
- All core algorithms GPU-accelerated:
  - Optimization (gradient descent)
  - MCMC (Metropolis-Hastings)
  - HMC/NUTS (gradient-guided)
  - SMC (particle methods)
  - Ensemble (emcee) [PR #797]
  - Nested sampling [PR #755]



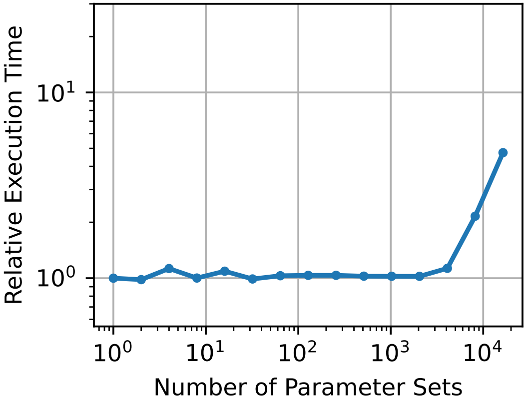
Framework Design

- Like numpy/scipy
- Not like cobaya/cosmosis
- Composable building blocks
- Maximum flexibility

Cosmology

GPU-Accelerated Applications: CMB and Cosmic Shear [2509.13307]

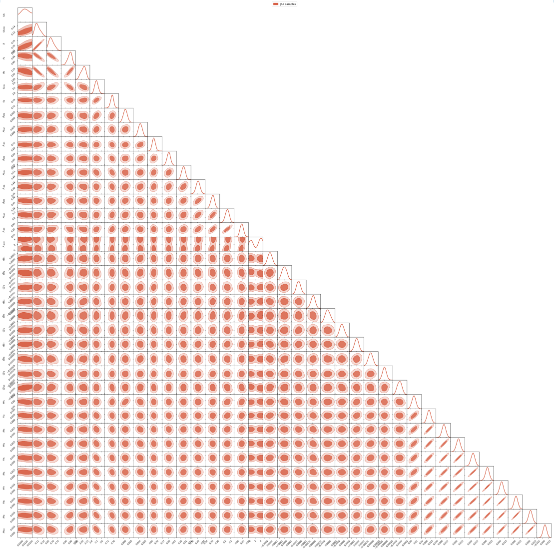
- **CMB (6 params)**: 300× speedup vs CPU PolyChord
- **Cosmic Shear (37 params)**: Days vs months (>1000× vs CPU; 10× vs GPU NUTS)
- **Method**: JAX neural emulators + GPU NS
- **Evidence**: Direct calculation with error bars
- **Models**: ΛCDM vs $w_0 w_a$ comparison



Cosmology

GPU-Accelerated Applications: CMB and Cosmic Shear [2509.13307]

- **CMB (6 params)**: 300× speedup vs CPU PolyChord
- **Cosmic Shear (37 params)**: Days vs months (>1000× vs CPU; 10× vs GPU NUTS)
- **Method**: JAX neural emulators + GPU NS
- **Evidence**: Direct calculation with error bars
- **Models**: ΛCDM vs $w_0 w_a$ comparison



The Vision: Full GPU Pipeline

DISCO-DJ → DISCO-CL [2311.03291]

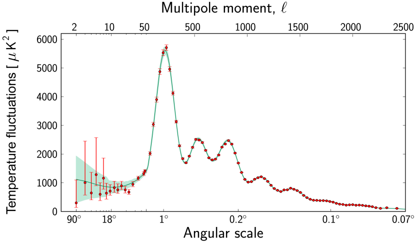
The Goal

Full CMB analysis that never leaves the GPU:

1. Einstein-Boltzmann solver → C_ℓ
2. CMB/LSS likelihoods
3. GPU nested sampling

DISCO-DJ/EB (Hahn+ 2024)

- Differentiable Einstein-Boltzmann in JAX
- Agrees with CLASS/CAMB at per-mille level
- Vectorised over cosmological parameters



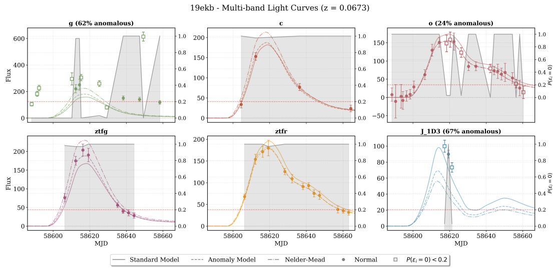
Current Work

Extending to CMB-ready C_ℓ with Oli Hahn & Nils Schöneberg

Type Ia Supernovae

GPU-Accelerated Applications: Bayesian Anomaly Detection [2509.13394]

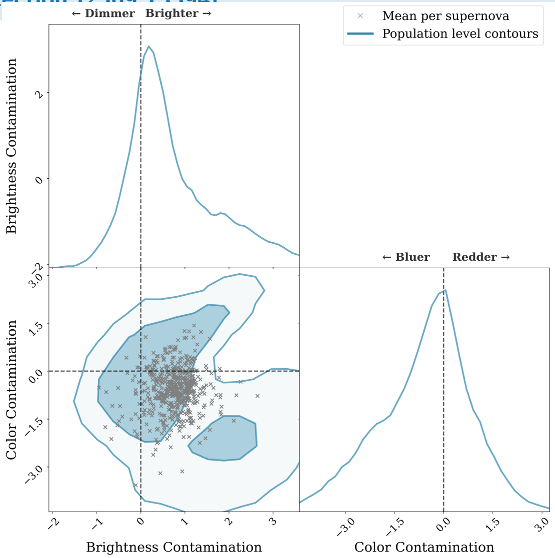
- **Problem**: Manual photometric rejection doesn't scale for LSST
- **Solution**: Bayesian anomaly detection integrated into SALT3 fitting
- **Method**: Model contamination probability per measurement
- **Result**: Automatic outlier/corrupted band rejection
- **Finding**: Contaminants systematically brighter/bluer
- **Impact**: Essential for unbiased cosmology at scale



Type Ia Supernovae

GPU-Accelerated Applications: Bayesian Anomaly Detection [2509.13394]

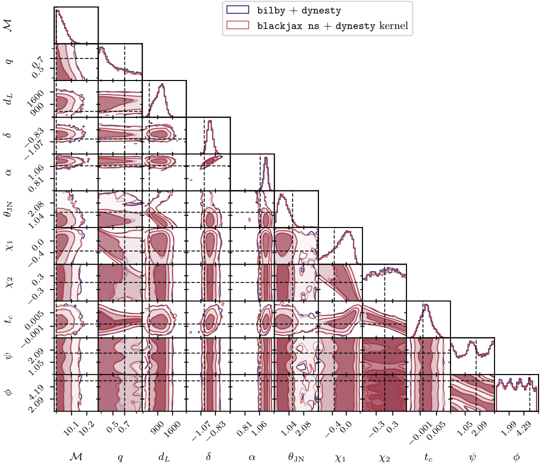
- **Problem**: Manual photometric rejection doesn't scale for LSST
- **Solution**: Bayesian anomaly detection integrated into SALT3 fitting
- **Method**: Model contamination probability per measurement
- **Result**: Automatic outlier/corrupted band rejection
- **Finding**: Contaminants systematically brighter/bluer
- **Impact**: Essential for unbiased cosmology at scale



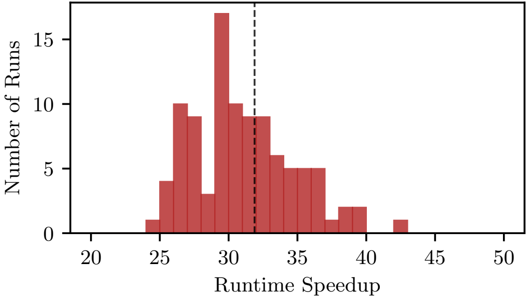
Gravitational Waves

GPU-Accelerated Applications: GPU-Native Nested Sampling [2509.04336]

- **Goal**: GPU-accelerate bilby's acceptance-walk NS
- **Implementation**: Faithful port to blackjax-ns
- **Performance**: 20–40× speedup for BBH
- **Validation**: Identical posteriors/evidences
- **Hardware**: Single GPU replaces CPU cluster
- **Impact**: Clean baseline for future methods



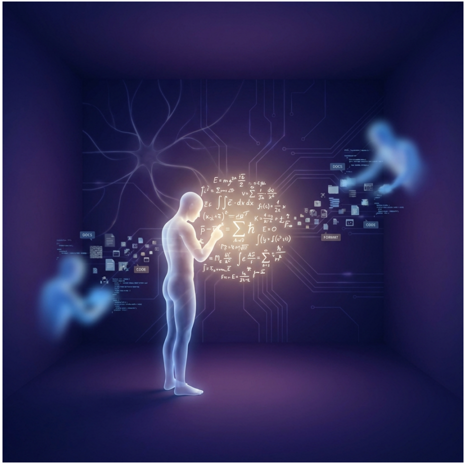
- **Goal**: GPU-accelerate bilby's acceptance-walk NS
- **Implementation**: Faithful port to blackjax-ns
- **Performance**: 20–40$\times$ speedup for BBH
- **Validation**: Identical posteriors/evidences
- **Hardware**: Single GPU replaces CPU cluster
- **Impact**: Clean baseline for future methods



The Second Revolution: AI for Scientists

Shifting gears: How did we build this code?

- Writing high-performance GPU code (CUDA, JAX) is historically difficult
- The "80/20" rule of scientific work:
  - 20% "hard thinking" (physics, algorithms)
  - 80% "boring tasks" (boilerplate, plotting, formatting)
- LLMs are not about "automating science" (the 20%)
- They collapse the 80% overhead, letting us focus on what matters
- BlackJAX NS: core algorithm hand-coded; matplotlib, tests, docs vibe-coded



Three Ways to Use AI in Research

1. **Foundation models**: Ask questions, get answers
   - Good for brainstorming
   - Limited for complex tasks
2. **Autonomous agents**: Let it run unsupervised
   - Generates volume, not quality
   - No accountability
3. **Supervised execution**: Human directs, AI executes
   - You provide intent and verify output
   - AI builds tools and infrastructure

What Works

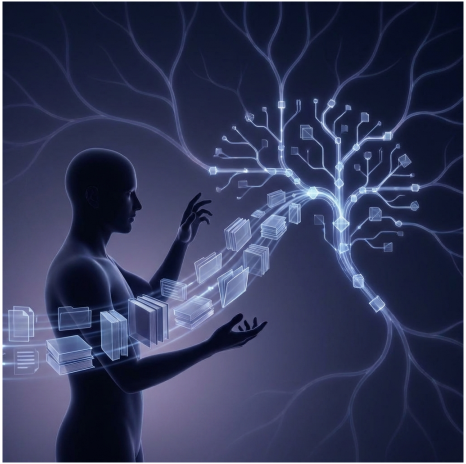
Human provides **intent** + **verification**. AI provides **implementation**.

From Prompt Engineering to Context Engineering

The skill of 2026

- 2024/25: "Prompt Engineering" (how to ask the question)
- 2026: "Context Engineering" (organising what the model knows)
- The bottleneck shifted:
  - Not "how to ask"
  - But "what information to provide"
- What works:
  - **Skills**: Read-only text files defining best practices (e.g. `cobaya.md`)
  - **MCPs**: Tools that allow AI to read/write files, run code, query databases

Knowledge → skill. Actions → MCP.



Case Study: Vibe Coding emcee into BlackJAX

15 minutes to a PR

The Problem

Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

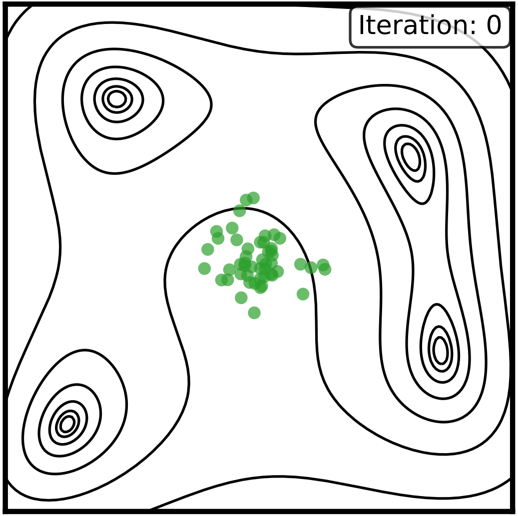
The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.

github.com/blackjax-devs/blackjax/pull/797



Case Study: Vibe Coding emcee into BlackJAX

15 minutes to a PR

The Problem

Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

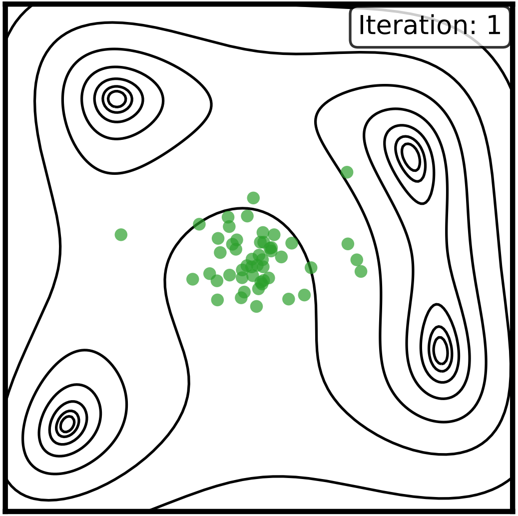
The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.

github.com/blackjax-devs/blackjax/pull/797



Case Study: Vibe Coding emcee into BlackJAX

15 minutes to a PR

The Problem

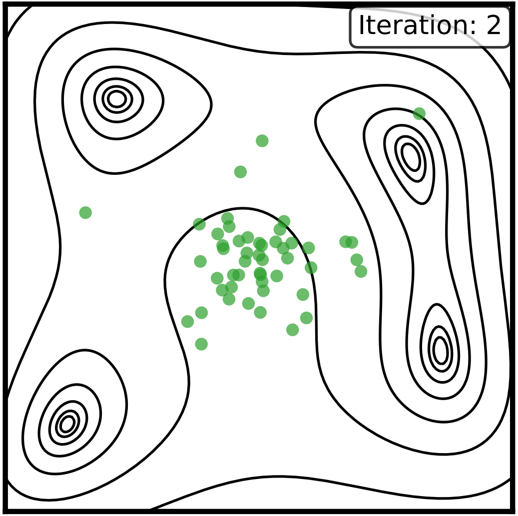
Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.
github.com/blackjax-devs/blackjax/pull/797



Case Study: Vibe Coding emcee into BlackJAX

15 minutes to a PR

The Problem

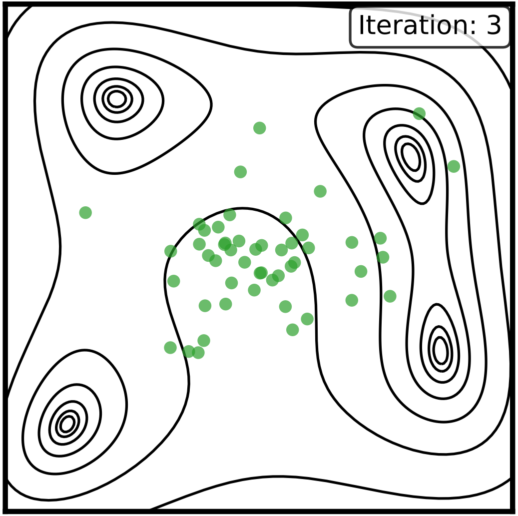
Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.
github.com/blackjax-devs/blackjax/pull/797



Case Study: Vibe Coding emcee into BlackJAX

15 minutes to a PR

The Problem

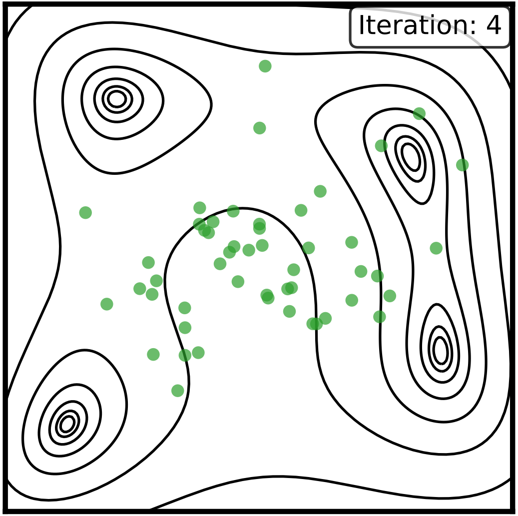
Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.
github.com/blackjax-devs/blackjax/pull/797



The Problem

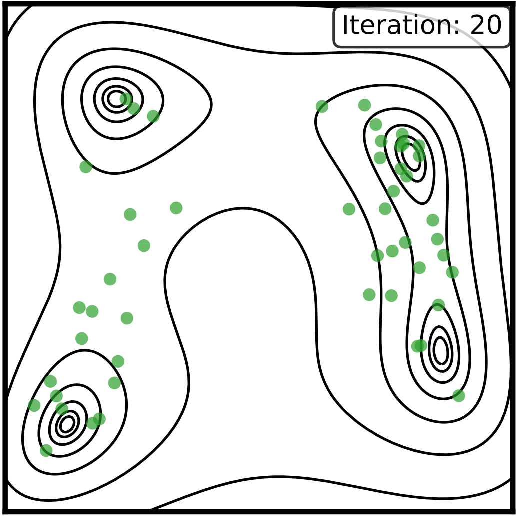
Needed an emcee animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The emcee paper [1202.3665]
- The emcee and BlackJAX repositories

Asked it to implement emcee in BlackJAX.
github.com/blackjax-devs/blackjax/pull/797



Case Study: Vibe Coding `emcee` into BlackJAX

15 minutes to a PR

The Problem

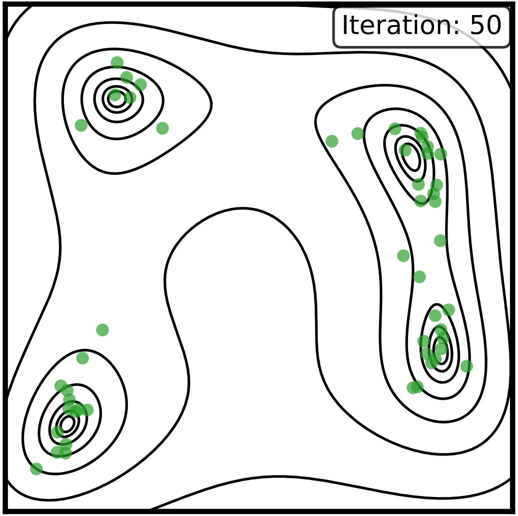
Needed an `emcee` animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The `emcee` paper [1202.3665]
- The `emcee` and BlackJAX repositories

Asked it to implement `emcee` in BlackJAX.
`github.com/blackjax-devs/blackjax/pull/797`



Case Study: Vibe Coding `emcee` into BlackJAX

15 minutes to a PR

The Problem

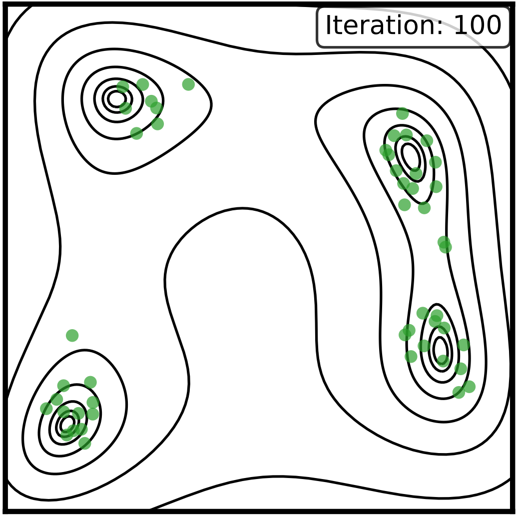
Needed an `emcee` animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

- The LaTeX of my talk
- The `emcee` paper [1202.3665]
- The `emcee` and BlackJAX repositories

Asked it to implement `emcee` in BlackJAX.
`github.com/blackjax-devs/blackjax/pull/797`



Case Study: Vibe Coding `emcee` into BlackJAX

15 minutes to a PR

The Problem

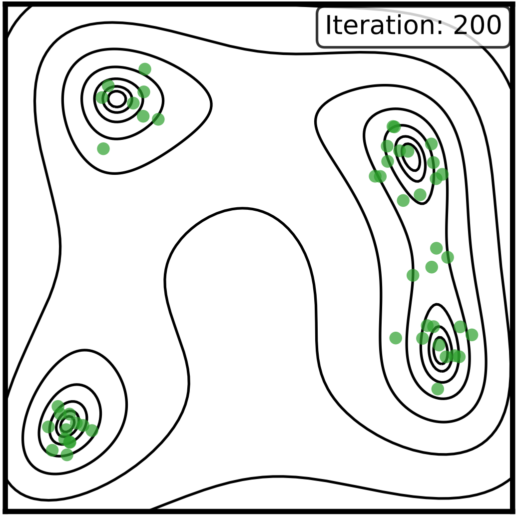
Needed an `emcee` animation equivalent to my nested sampling one. No BlackJAX implementation existed.

The Solution

Gave Claude Code:

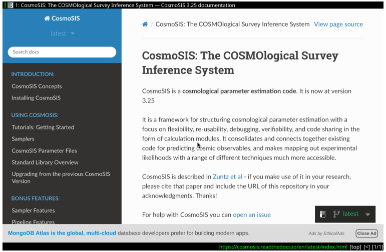
- The LaTeX of my talk
- The `emcee` paper [1202.3665]
- The `emcee` and BlackJAX repositories

Asked it to implement `emcee` in BlackJAX.
`github.com/blackjax-devs/blackjax/pull/797`



Cosmosis to Cobaya

Case Study: AI-Assisted Development



$\downarrow$

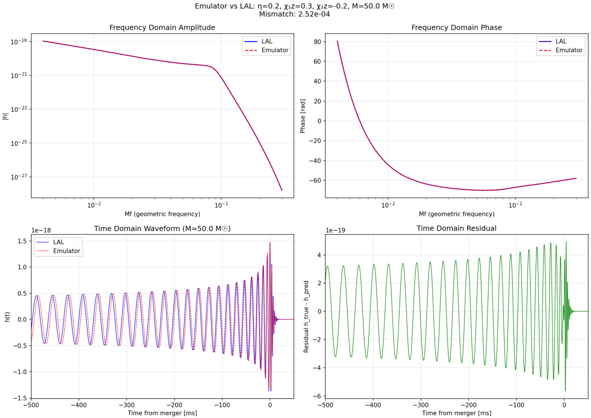
- **Task**: Port the DES Y3 likelihood to Cobaya
- State-of-the-art, but locked into Cosmosis
- Need it in Cobaya to combine consistently with CMB
- AI drafts the code; human defines the tests
- "Must match to floating-point precision"
- The tool works because the tests are strict

jim-emulators

Case Study: AI-Assisted Development (with James Alvey)

- **Goal**: JAX emulator infrastructure for GW
- Started at 9am with nothing
- By 6pm: fully working, documented, tested package
- We didn't feel tired. We felt invigorated.

github.com/handley-lab/jim-emulators

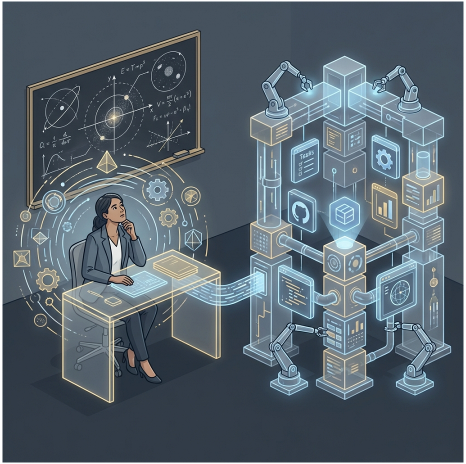


The Vision: Solving “Code Debt”

Cosmology is riven with technical debt

- PhD students spend years learning to wrestle with `Cobaya`, `Cosmosis`, or `MontePython`
- These are massive, complex codes
- Hard for PhDs to do anything profound when the entry barrier is so high
- The future:
  - Skills (text descriptions of physics) become the source of truth
  - Not the Fortran code written in 1998
  - Smaller, modular scripts generated on-demand

Skills marketplace:
`github.com/fundamental-physics/marketplace`



Summary

github.com/handley-lab/group

1. **GPU $\neq$ Machine Learning: Two Independent Capabilities**
   - GPUs accelerate any parallel algorithm
   - Automatic differentiation + massive parallelization — often confused, serve different purposes
2. **Classical Algorithms on GPU Competitive with ML State of the Art**
   - Traditional physics methods + GPU = superior performance
   - 300$\times$ speedup for CMB, 20-40$\times$ for gravitational waves
3. **AI Accelerates Development as well as Computation**
   - LLMs solve the GPU porting challenge at scale
   - Context engineering is the emerging skill

Get Started with GPU-Accelerated Sampling

handley-lab.co.uk/nested-sampling-book

Appendix: Sampling Methods in Detail

Pedagogical animations for reference

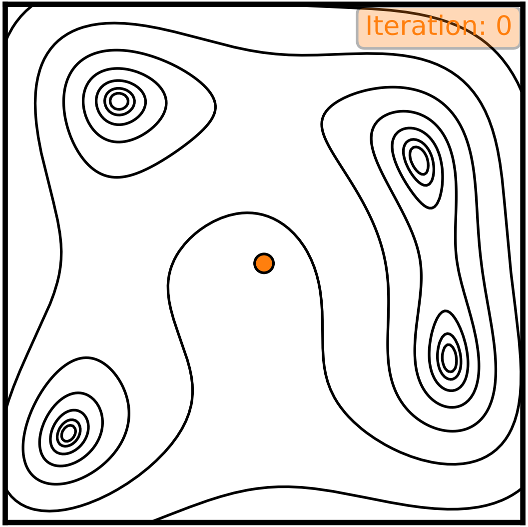
The following slides provide detailed animations of sampling methods discussed in the main talk.

The Simplest Approach: Optimization (e.g., χ^2 Minimization)

How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is "downhill".

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.



Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.

The Simplest Approach: Optimization (e.g., χ^2 Minimization)

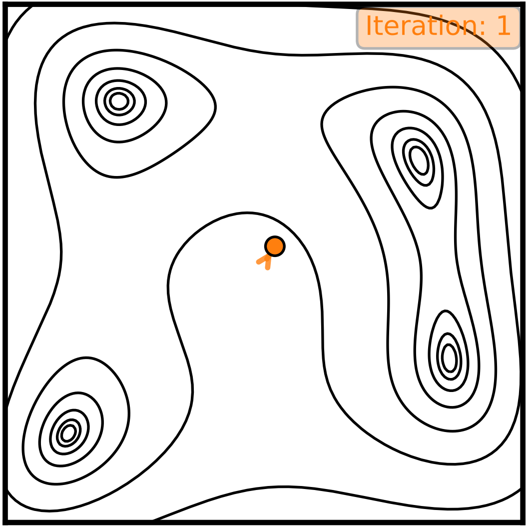
How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is "downhill".

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.

Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.



The Simplest Approach: Optimization (e.g., χ^2 Minimization)

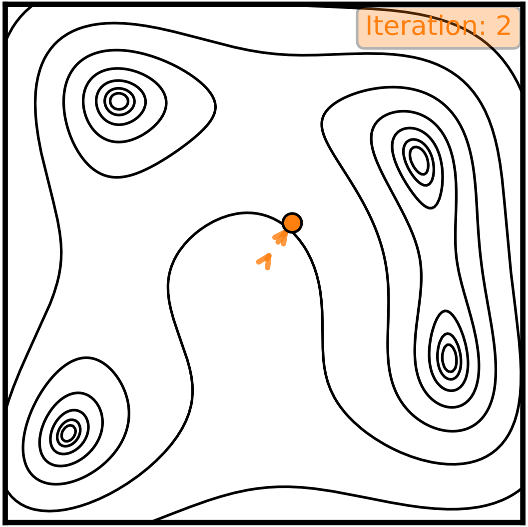
How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is "downhill".

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.

Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.



The Simplest Approach: Optimization (e.g., χ^2 Minimization)

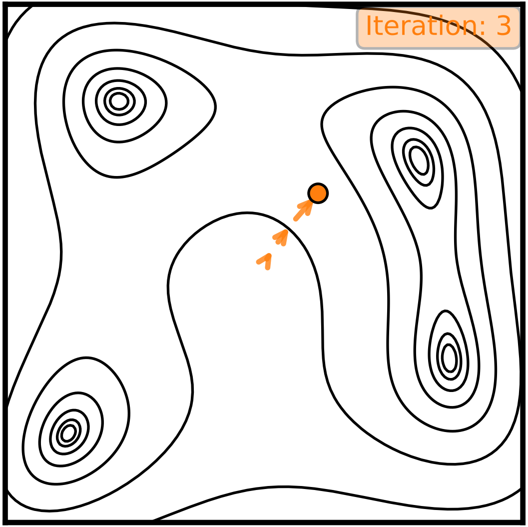
How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is "downhill".

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.

Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.



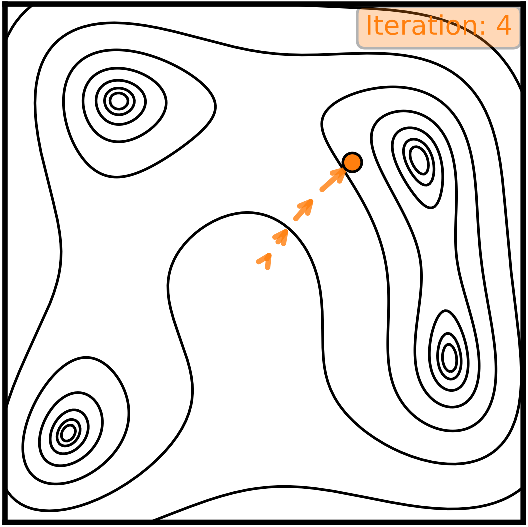
How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is "downhill".

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.

Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.



The Simplest Approach: Optimization (e.g., χ^2 Minimization)

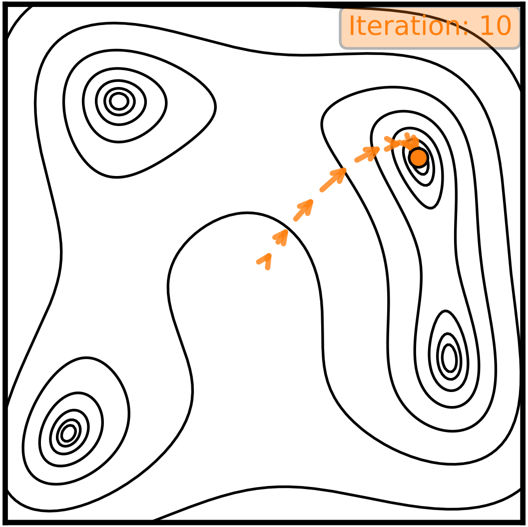
How it Works: Hill Climbing

Imagine the parameter space is a landscape where lower χ^2 (or higher likelihood) is “downhill”.

- Start somewhere.
- Follow the steepest gradient downhill.
- Stop when you reach the bottom of a valley.

Advantages

- **Fast** and computationally cheap.
- Good for a quick first look.



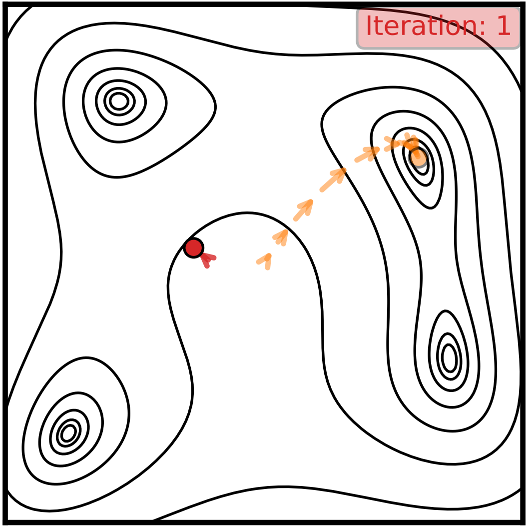
The Simplest Approach: Optimization (e.g., χ^2 Minimization)

Limitations

- Only gives a single **point estimate** (the "best fit").
- **No uncertainty quantification!** Where are the error bars?
- Can easily get stuck in a **local minimum**, missing the true global best fit.

Key Message

Optimization is fast but gives an incomplete and potentially misleading picture. Science needs error bars.



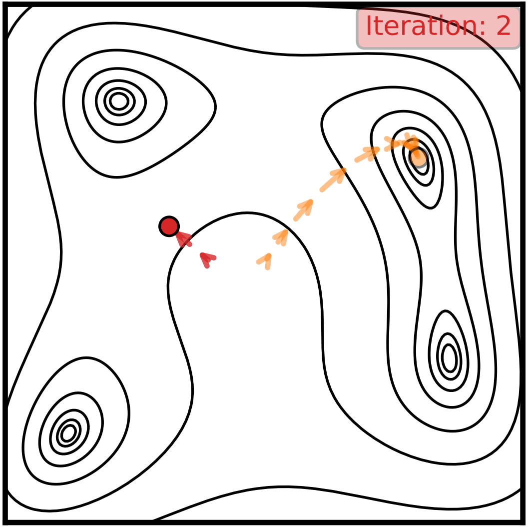
The Simplest Approach: Optimization (e.g., χ^2 Minimization)

Limitations

- Only gives a single **point estimate** (the "best fit").
- **No uncertainty quantification!** Where are the error bars?
- Can easily get stuck in a **local minimum**, missing the true global best fit.

Key Message

Optimization is fast but gives an incomplete and potentially misleading picture. Science needs error bars.



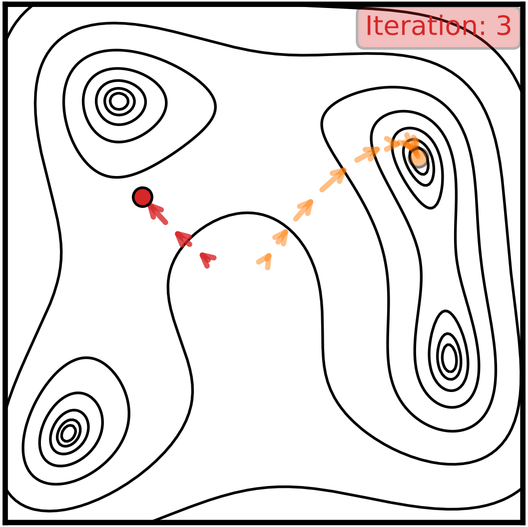
The Simplest Approach: Optimization (e.g., χ^2 Minimization)

Limitations

- Only gives a single **point estimate** (the "best fit").
- **No uncertainty quantification!** Where are the error bars?
- Can easily get stuck in a **local minimum**, missing the true global best fit.

Key Message

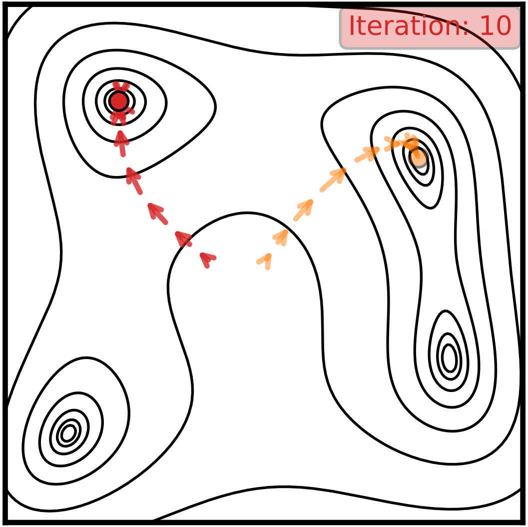
Optimization is fast but gives an incomplete and potentially misleading picture. Science needs error bars.



The Simplest Approach: Optimization (e.g., χ^2 Minimization)

Limitations

- Only gives a single **point estimate** (the "best fit").
- **No uncertainty quantification!** Where are the error bars?
- Can easily get stuck in a **local minimum**, missing the true global best fit.



Key Message

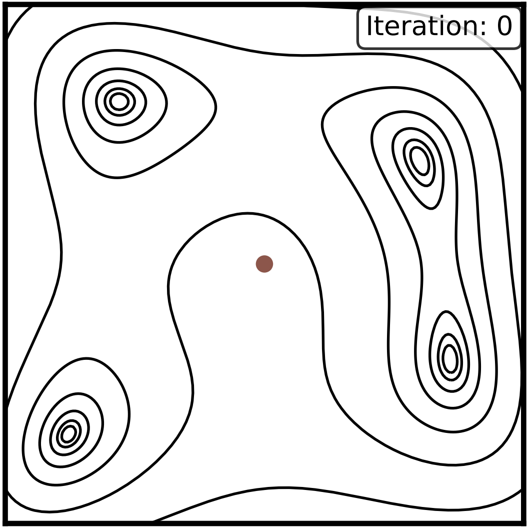
Optimization is fast but gives an incomplete and potentially misleading picture. Science needs error bars.

The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

How it Works (Metropolis-Hastings)

Imagine a “walker” exploring the parameter landscape.

1. Take a random step to a new position.
2. If the new spot is “higher” (better likelihood), move there.
3. If it’s “lower”, maybe move there anyway (with probability proportional to how much lower it is).
4. Repeat millions of times. The path the walker takes traces the posterior distribution.

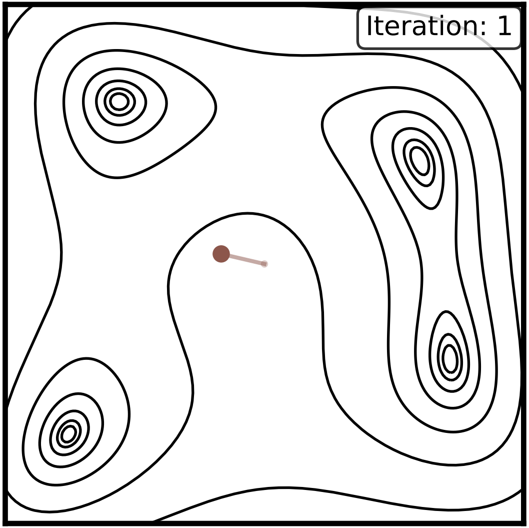


The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

How it Works (Metropolis-Hastings)

Imagine a "walker" exploring the parameter landscape.

1. Take a random step to a new position.
2. If the new spot is "higher" (better likelihood), move there.
3. If it's "lower", maybe move there anyway (with probability proportional to how much lower it is).
4. Repeat millions of times. The path the walker takes traces the posterior distribution.

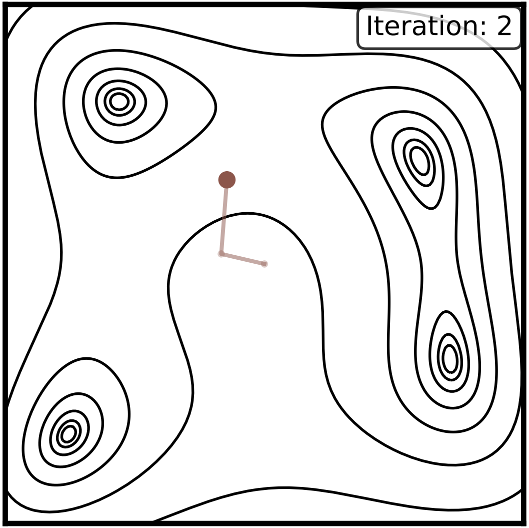


The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

How it Works (Metropolis-Hastings)

Imagine a "walker" exploring the parameter landscape.

1. Take a random step to a new position.
2. If the new spot is "higher" (better likelihood), move there.
3. If it's "lower", maybe move there anyway (with probability proportional to how much lower it is).
4. Repeat millions of times. The path the walker takes traces the posterior distribution.

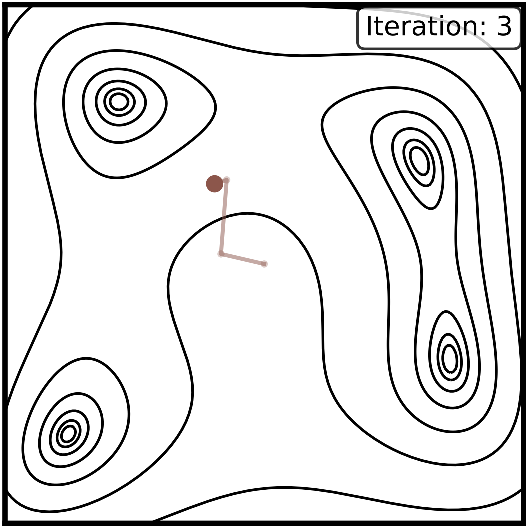


The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

How it Works (Metropolis-Hastings)

Imagine a "walker" exploring the parameter landscape.

1. Take a random step to a new position.
2. If the new spot is "higher" (better likelihood), move there.
3. If it's "lower", maybe move there anyway (with probability proportional to how much lower it is).
4. Repeat millions of times. The path the walker takes traces the posterior distribution.

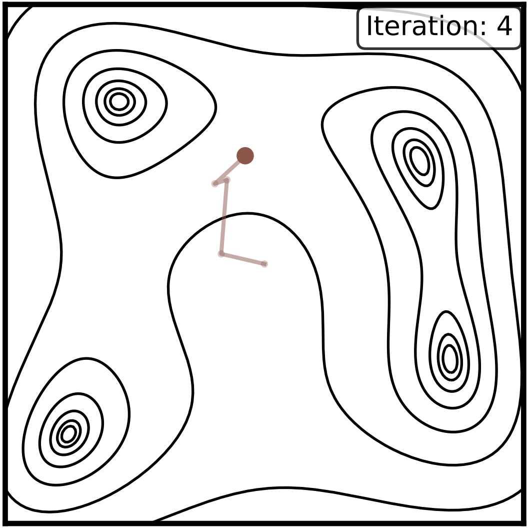


The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

How it Works (Metropolis-Hastings)

Imagine a “walker” exploring the parameter landscape.

1. Take a random step to a new position.
2. If the new spot is “higher” (better likelihood), move there.
3. If it’s “lower”, maybe move there anyway (with probability proportional to how much lower it is).
4. Repeat millions of times. The path the walker takes traces the posterior distribution.



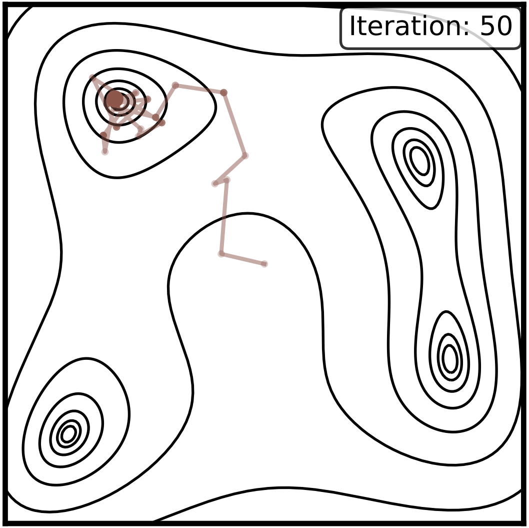
The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

Advantages & Limitations

- Explores the full posterior and gives uncertainties.
- Limitation: The walker can be inefficient. It can get "stuck" in a local high-likelihood region and fail to find other, separate modes.
- Limitation: Can be slow to explore highly correlated ("banana-shaped") posteriors.

Key Message

MCMC is a foundational sampling method, but its simple "random walk" can be inefficient in the complex parameter spaces of astronomical inference.



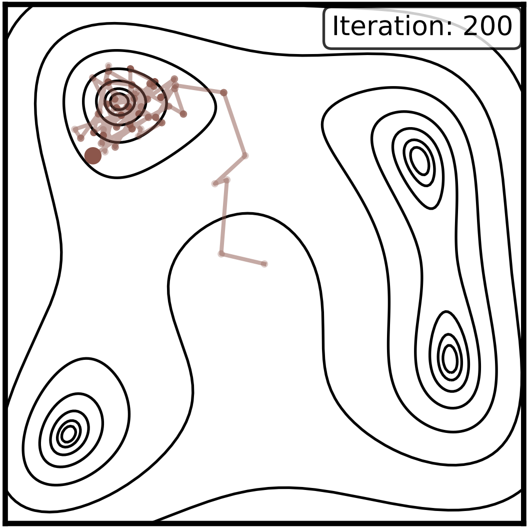
The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

Advantages & Limitations

- Explores the full posterior and gives uncertainties.
- Limitation: The walker can be inefficient. It can get "stuck" in a local high-likelihood region and fail to find other, separate modes.
- Limitation: Can be slow to explore highly correlated ("banana-shaped") posteriors.

Key Message

MCMC is a foundational sampling method, but its simple "random walk" can be inefficient in the complex parameter spaces of astronomical inference.



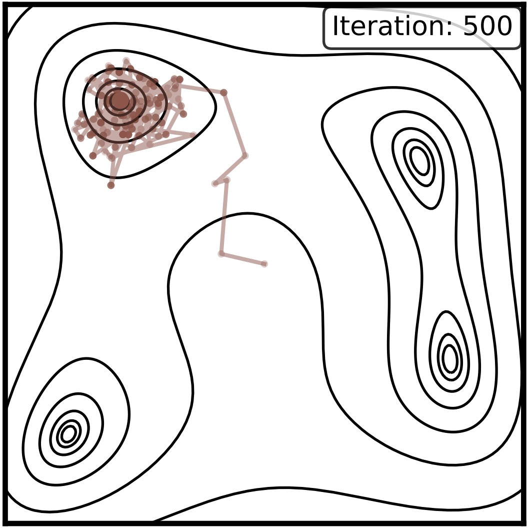
The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

Advantages & Limitations

- Explores the full posterior and gives uncertainties.
- Limitation: The walker can be inefficient. It can get "stuck" in a local high-likelihood region and fail to find other, separate modes.
- Limitation: Can be slow to explore highly correlated ("banana-shaped") posteriors.

Key Message

MCMC is a foundational sampling method, but its simple "random walk" can be inefficient in the complex parameter spaces of astronomical inference.



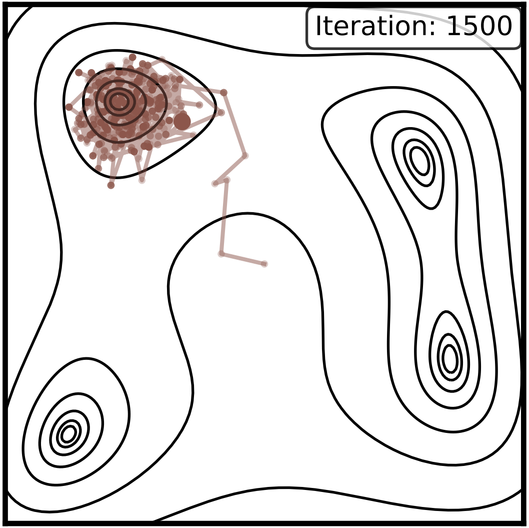
The Classic Workhorse: Markov Chain Monte Carlo (MCMC)

Advantages & Limitations

- Explores the full posterior and gives uncertainties.
- Limitation: The walker can be inefficient. It can get "stuck" in a local high-likelihood region and fail to find other, separate modes.
- Limitation: Can be slow to explore highly correlated ("banana-shaped") posteriors.

Key Message

MCMC is a foundational sampling method, but its simple "random walk" can be inefficient in the complex parameter spaces of astronomical inference.

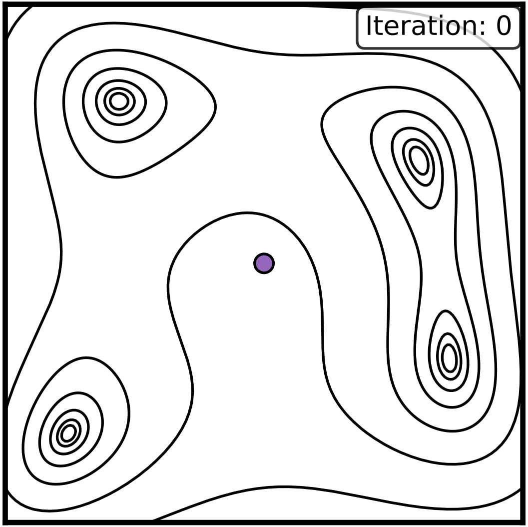


Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

How it Works

Uses gradients to guide exploration more efficiently than random walks.

1. Treat parameters as "particles" with position and momentum.
2. Use gradient of log-likelihood as "force" to guide movement.
3. Propose coherent moves along gradient directions.
4. Accept/reject using Metropolis criterion.

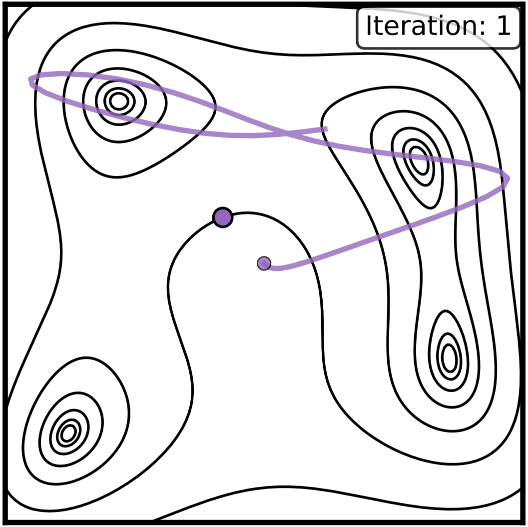


Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

How it Works

Uses gradients to guide exploration more efficiently than random walks.

1. Treat parameters as "particles" with position and momentum.
2. Use gradient of log-likelihood as "force" to guide movement.
3. Propose coherent moves along gradient directions.
4. Accept/reject using Metropolis criterion.

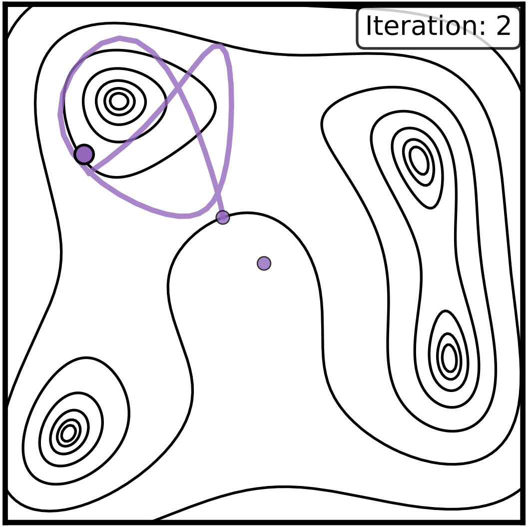


Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

How it Works

Uses gradients to guide exploration more efficiently than random walks.

1. Treat parameters as "particles" with position and momentum.
2. Use gradient of log-likelihood as "force" to guide movement.
3. Propose coherent moves along gradient directions.
4. Accept/reject using Metropolis criterion.

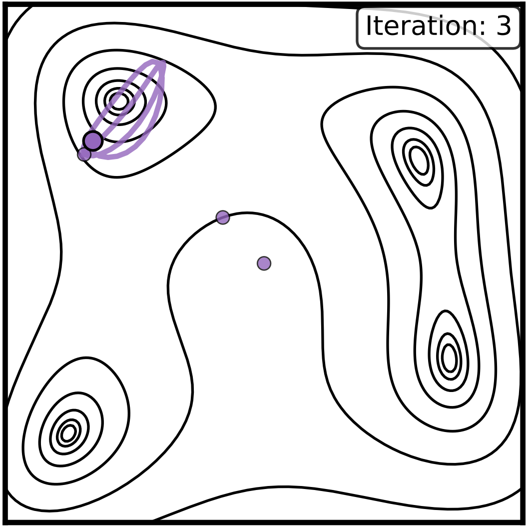


Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

How it Works

Uses gradients to guide exploration more efficiently than random walks.

1. Treat parameters as "particles" with position and momentum.
2. Use gradient of log-likelihood as "force" to guide movement.
3. Propose coherent moves along gradient directions.
4. Accept/reject using Metropolis criterion.

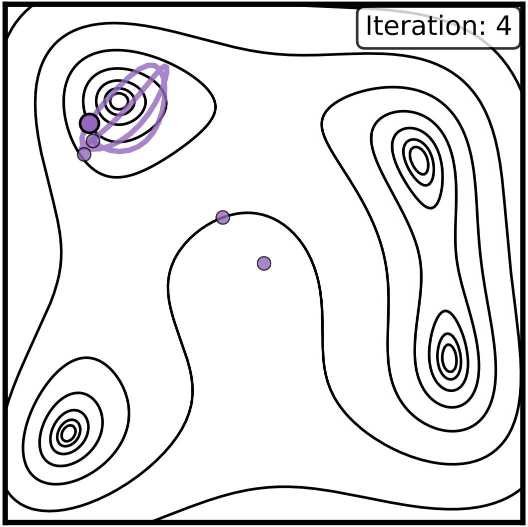


Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

How it Works

Uses gradients to guide exploration more efficiently than random walks.

1. Treat parameters as "particles" with position and momentum.
2. Use gradient of log-likelihood as "force" to guide movement.
3. Propose coherent moves along gradient directions.
4. Accept/reject using Metropolis criterion.



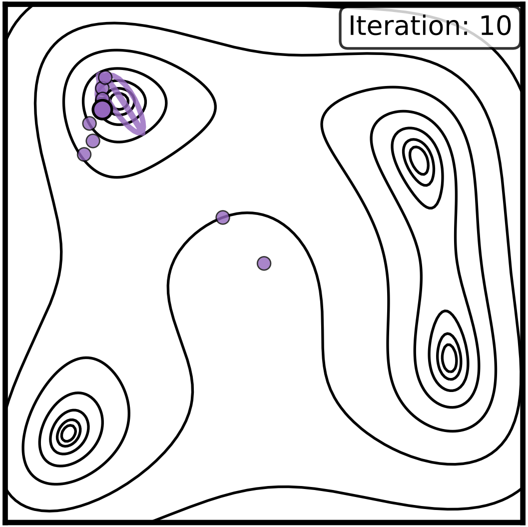
Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

Advantages & Requirements

- Much more efficient than random walk for smooth posteriors.
- Requires gradients of the likelihood function.
- Can traverse parameter space much faster.
- Less likely to get stuck in local regions.

Key Message

HMC leverages gradient information for efficient sampling, but requires differentiable models.



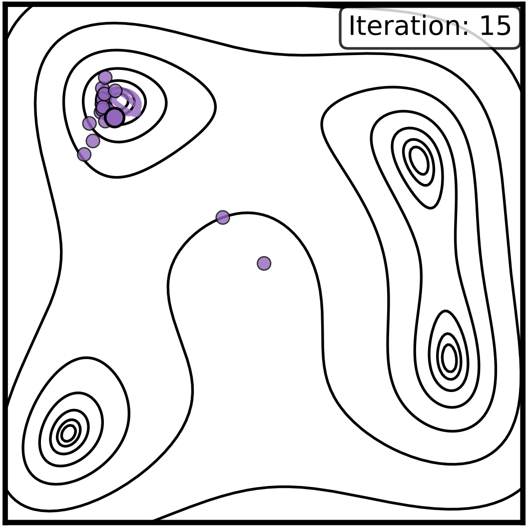
Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

Advantages & Requirements

- Much more efficient than random walk for smooth posteriors.
- Requires gradients of the likelihood function.
- Can traverse parameter space much faster.
- Less likely to get stuck in local regions.

Key Message

HMC leverages gradient information for efficient sampling, but requires differentiable models.



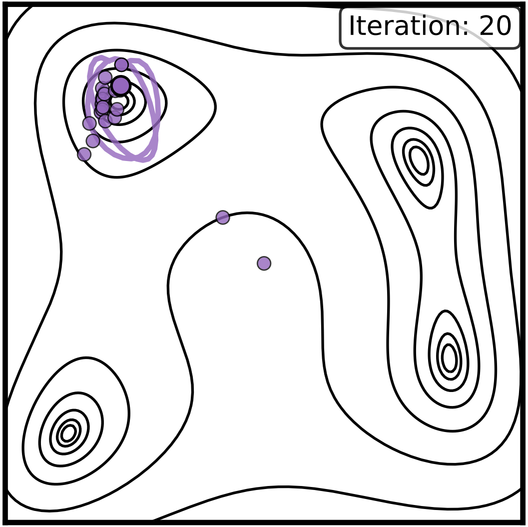
Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

Advantages & Requirements

- Much more efficient than random walk for smooth posteriors.
- Requires gradients of the likelihood function.
- Can traverse parameter space much faster.
- Less likely to get stuck in local regions.

Key Message

HMC leverages gradient information for efficient sampling, but requires differentiable models.



Gradient-Guided Sampling: Hamiltonian Monte Carlo (HMC)

Advantages & Requirements

- Much more efficient than random walk for smooth posteriors.
- Requires gradients of the likelihood function.
- Can traverse parameter space much faster.
- Less likely to get stuck in local regions.

Key Message

HMC leverages gradient information for efficient sampling, but requires differentiable models.